

Universidad Gerardo Barrios
Facultad de Ciencia y Tecnología



Informe Final de Proyecto Trabajo de Graduación
Modalidad Preespecialización en la Especialidad de:
Desarrollo de Aplicaciones con Inteligencia Artificial
“EduBot AI – Micro-aprendizaje Evolutivo (API Inteligente)”

Presentado por:

Gabriel Eduardo Henríquez González

Melvin Josué Pereira Amaya

Alexis Manuel Calix Magaña

Índice

1. Resumen del Proyecto	3
2. Marco Conceptual	4
2.1. Inteligencia Artificial en la Educación (EdTech)	4
2.2. Modelos de Lenguaje de Gran Tamaño (LLMs) y Llama 3	4
2.3. Arquitectura Orientada a Microservicios y API REST (FastAPI)	5
2.4. MLOps: Operaciones de Machine Learning (Nivel 0)	6
2.5. Concurrencia y Asincronismo en Python	7
2.6. Infraestructura, Contenedores y Plataformas Cloud (PaaS)	8
3. Análisis del Problema	10
4. Propuesta de Solución	11
5. Metodología de Desarrollo del Proyecto	12
6. Resultados Alcanzados	15
7. Conclusiones Generales	17
8. Referencias Estudiadas	18

1. Resumen del proyecto

El presente proyecto de graduación detalla el diseño, desarrollo, implementación y despliegue de **EdubotAI** (versión 1.1.0), un tutor virtual inteligente estructurado como una Interfaz de Programación de Aplicaciones (API) REST, accesible a través de la plataforma de mensajería Telegram. El propósito central de esta solución es mitigar la alta tasa de deserción en entornos de aprendizaje autónomo de ciencias de la computación, proveyendo a los estudiantes de retroalimentación personalizada, instantánea y conversacional, operando como un motor cognitivo 24/7.

A nivel arquitectónico, el sistema fue desarrollado utilizando el framework de alto rendimiento FastAPI (Python), garantizando validaciones estrictas de datos mediante esquemas de Pydantic. Para el componente de Inteligencia Artificial, se integró el Modelo de Lenguaje de Gran Tamaño (LLM) llama-3.3-70b-versatile a través del proveedor de infraestructura en la nube Groq Cloud, lo que permite latencias ultrabajas en el procesamiento de Lenguaje Natural (NLP). Adicionalmente, el ecosistema cuenta con un modelo predictivo estático (serializado en formato Pickle) para el análisis de riesgo de deserción del alumno.

El desarrollo técnico del proyecto atravesó rigurosas fases de ingeniería de software y operaciones (DevOps / MLOps). Se implementaron flujos de Integración y Entrega Continua (CI/CD) mediante GitHub Actions, pruebas unitarias automatizadas y contenedorización inmutable a través de Docker. En la capa de observabilidad, se desarrolló un *Middleware* global que inyecta identificadores de trazabilidad (`request_id`) y captura métricas operativas de latencia (percentiles p50 y p95) sin comprometer la privacidad de los datos sensibles de los usuarios.

Para garantizar la viabilidad y escalabilidad del producto en un entorno de producción, la versión final (Release Candidate v1.1.0) incorpora estrategias de optimización avanzadas: un sistema de semáforos asíncronos (`asyncio.Semaphore`) que permite gestionar hasta 40 usuarios concurrentes sin degradación del servicio, y un patrón de caché en memoria (`lru_cache`) acoplado a un límite estricto de tokens de salida. Esto asegura tiempos de respuesta casi instantáneos ante consultas

repetitivas y protege las cuotas del proveedor externo (Rate Limits). Finalmente, el producto fue desplegado de manera exitosa y pública en la plataforma Render, demostrando un nivel de madurez operativa (MLOps Nivel 0 con DevOps avanzado) capaz de ser auditado, versionado y operado de manera profesional.

2. Marco Conceptual

El desarrollo de sistemas de software integrados con Inteligencia Artificial requiere la convergencia de múltiples disciplinas de las ciencias de la computación. A continuación, se exponen los fundamentos teóricos que sustentan la arquitectura y operación de EdubotAI.

2.1. Inteligencia Artificial en la Educación (EdTech)

La Inteligencia Artificial (IA) aplicada a la educación ha evolucionado desde simples sistemas de tutoría basados en reglas, hacia Motores Cognitivos capaces de interpretar intenciones, contextos y ambigüedades. Los Sistemas de Tutoría Inteligente (STI) modernos simulan el comportamiento de un educador humano, brindando instrucción personalizada (1:1). Según la teoría del micro-aprendizaje evolutivo, la asimilación del conocimiento mejora significativamente cuando los estudiantes reciben retroalimentación inmediata sobre conceptos atómicos (como variables, bucles o estructuras de datos), en lugar de consumir largas sesiones de contenido estático. En este contexto, el procesamiento de Lenguaje Natural (NLP) es el pilar que permite al estudiante interactuar con la máquina utilizando su propio vocabulario, reduciendo la fricción tecnológica y aumentando la retención escolar.

2.2. Modelos de Lenguaje de Gran Tamaño (LLMs) y Llama 3

Los LLMs (Large Language Models) son redes neuronales basadas en la arquitectura *Transformer*, entrenadas con conjuntos masivos de texto para predecir, generar y comprender lenguaje natural con un nivel de coherencia cercano al humano.

En el núcleo de EdubotAI se utiliza **Llama 3** (específicamente la variante llama-3.3-70b-versatile), un modelo fundacional de código abierto desarrollado por Meta. La elección de este modelo radica en su equilibrio excepcional entre capacidad de razonamiento lógico (ideal para enseñar ciencias de la computación y programación) y eficiencia computacional.

Para interactuar con este modelo sin los costos prohibitivos de poseer hardware dedicado (como clústeres de GPUs A100), el proyecto utiliza Inferencia como Servicio (MaaS) a través de **Groq Cloud**. Groq utiliza Unidades de Procesamiento Tensorial del Lenguaje (LPUs), un hardware diseñado exclusivamente para inferencia determinista que elimina los cuellos de botella de memoria que sufren las GPUs tradicionales, logrando generar cientos de *tokens* (fragmentos de palabras) por segundo. Esto resuelve el problema clásico de la latencia en los chatbots educativos.

2.3. Arquitectura Orientada a Microservicios y API REST (FastAPI)

Para construir un puente entre el cliente (Telegram) y el motor de IA, se requiere una interfaz de programación robusta. Una API REST (Representational State Transfer) permite la comunicación de sistemas a través del protocolo HTTP, utilizando métodos estandarizados (POST, GET) e intercambiando información en formato JSON.

EdubotAI está construido sobre **FastAPI**, un framework web moderno para Python 3.8+ basado en las sugerencias de tipos estándar de Python. Las ventajas teóricas de FastAPI incluyen:

- **Alto Rendimiento:** Comparable a Node.js y Go, gracias a su motor interno basado en Starlette y Pydantic.
- **Validación de Esquemas (Pydantic):** Garantiza que los datos que ingresan al sistema (como el estudiante_id o el modulo) sean estrictamente del tipo de dato correcto. Si el estudiante envía un dato erróneo, la API rechaza la solicitud instantáneamente (Error 400 Bad Request) antes de que el error llegue al motor de IA, ahorrando tiempo de cómputo.
- **Documentación Automática:** Genera interfaces Swagger UI que permiten la

experimentación y validación del contrato de la API en tiempo real.

2.4. MLOps: Operaciones de Machine Learning (Nivel 0)

MLOps (Machine Learning Operations) es una práctica de ingeniería de software que busca unificar el desarrollo de sistemas de aprendizaje automático (ML) y la operación de los mismos. Es el equivalente a aplicar prácticas de DevOps a los sistemas de datos e IA.

Según el marco de referencia de Google Cloud, existen diferentes niveles de madurez en MLOps. EdubotAI se clasifica arquitectónicamente en el **Nivel 0: Proceso Manual**. En este nivel, las fases experimentales (entrenamiento del modelo de predicción de deserción, serialización en archivo .pkl o *Pickle*) ocurren fuera del sistema principal, y se realiza un traspaso manual del modelo hacia la ingeniería de software (FastAPI).

Aunque a nivel puramente predictivo el sistema es de Nivel 0, el ecosistema circundante posee un **DevOps Avanzado**. La aplicación de IA requiere prácticas estrictas como:

- **Observabilidad:** Capacidad de entender el estado interno del sistema leyendo sus salidas externas (Logs). En EdubotAI, se registran eventos como `request_completed` e `ai_inference` acoplados a un `request_id` (UUIDv4) para trazabilidad.
- **Línea Base de Rendimiento:** Medición matemática del comportamiento del software mediante percentiles. El p50 (mediana) indica el tiempo típico de respuesta, mientras que el p95 revela la "cola lenta" (el 5% de los usuarios que experimentan mayor latencia), permitiendo aislar si el cuello de botella es el código local o la latencia de red del proveedor externo (Grog).

2.5. Concurrencia y Asincronismo en Python

Un problema fundamental en sistemas web tradicionales (síncronos) es el bloqueo por Entrada/Salida (I/O). Si un sistema síncrono llama a una IA externa que tarda 2

segundos en responder, todo el servidor se congela durante esos 2 segundos. Python moderno resuelve esto mediante la librería `asyncio`, la cual permite un modelo de ejecución concurrente de un solo hilo. Cuando FastAPI hace un llamado asíncrono (usando `await`), cede el control temporalmente para atender a otro usuario mientras espera la respuesta de la IA.

Para proteger al sistema y al proveedor externo del agotamiento de recursos (Error 429: *Too Many Requests*), la teoría de sistemas distribuidos sugiere el uso de primitivas de sincronización. Un **Semáforo Asíncrono** es un objeto que mantiene un contador interno; si el límite es 40, deja pasar a 40 usuarios simultáneamente. El usuario número 41 es puesto en una cola de espera en milisegundos hasta que un espacio se libere, previniendo la caída del servidor (DDoS accidental) y garantizando la resiliencia operativa.

Asimismo, la **Caché en Memoria** (`lru_cache - Least Recently Used`) fundamenta la optimización algorítmica: si la respuesta a una misma consulta ya fue calculada (o generada por el LLM), se almacena temporalmente. Solicitudes idénticas futuras consumen 0 recursos de red y 0 tokens, resolviéndose en fracciones de milisegundo ($O(1)$ en complejidad de tiempo).

2.6. Infraestructura, Contenedores y Plataformas Cloud (PaaS)

Para eliminar el problema de "funciona en mi computadora", se emplea la contenedorización. **Docker** empaqueta el código fuente, el entorno de ejecución (Python 3.11), las librerías del sistema y las dependencias (descritas en un `requirements.txt`) en una "Imagen" inmutable. Esta imagen se ejecuta como un contenedor aislado en cualquier sistema operativo.

Finalmente, el alojamiento se gestiona mediante arquitecturas PaaS (Platform as a Service) como **Render**, que proveen un servidor gestionado que automáticamente construye el contenedor a partir de un repositorio de GitHub, inyecta variables de entorno seguras (sin exponer los secretos o Tokens) y expone un puerto público hacia la red global (Internet), cerrando el ciclo de Integración y Entrega Continua (CI/CD).

2.7. Procesamiento de Lenguaje Natural (PLN) y Arquitecturas Subyacentes

El Procesamiento de Lenguaje Natural (PLN) es la rama de la Inteligencia Artificial dedicada a dotar a las computadoras de la capacidad de entender, interpretar y manipular el lenguaje humano. En el contexto de EdubotAI, el PLN no se limita a la simple coincidencia de palabras clave (keyword matching), sino que emplea análisis semántico y sintáctico profundo.

El proceso fundamental detrás de la interpretación del lenguaje por parte de los LLMs (como Llama 3) comienza con la "Tokenización". Un token es la unidad mínima de procesamiento (puede ser una palabra, una sílaba o un carácter). El modelo convierte el texto del estudiante en vectores matemáticos de alta dimensionalidad (Embeddings). Estos vectores permiten que la máquina comprenda que palabras como "bucle", "ciclo" y "loop" tienen una proximidad semántica casi idéntica en el contexto de la programación, permitiendo al tutor virtual responder con precisión sin importar las variaciones del lenguaje coloquial o técnico que utilice el alumno. Esta abstracción matemática es lo que dota al sistema de su capacidad de razonamiento aparentemente cognitivo.

2.8. Ingeniería de Prompts (Prompt Engineering) y Parametrización Estocástica

La integración de un Modelo de Lenguaje en una aplicación educativa requiere un control estricto sobre su comportamiento, lo cual se logra mediante la Ingeniería de Prompts. Esta disciplina consiste en el diseño estructurado de las instrucciones (System Prompts) que definen la "personalidad", los límites éticos y el formato de respuesta de la IA.

En la arquitectura de EdubotAI, el LLM opera bajo un contexto predefinido que le instruye actuar como un tutor de ciencias de la computación. Además del texto, la generación de respuestas se controla mediante parámetros estocásticos (probabilísticos):

Temperatura (Temperature): Controla la aleatoriedad de las predicciones. Una temperatura cercana a 0.0 produce respuestas deterministas y repetitivas, mientras que una cercana a 1.0 genera textos creativos. Para un entorno educativo de programación, se requiere un balance (ej. 0.5) para que las respuestas sean exactas pero conversacionalmente amigables.

Max_Tokens: Este parámetro es vital para la protección de recursos (Rate Limits) y para forzar el modelo de micro-aprendizaje. Al limitar matemáticamente la salida (por ejemplo, a 150 tokens), se obliga a la red neuronal a sintetizar la información, entregando respuestas atómicas y directas que evitan la sobrecarga cognitiva del estudiante.

2.9. Serialización de Modelos Tradicionales (Machine Learning) y Formato Pickle

Mientras que el procesamiento de lenguaje se delega a la nube, el sistema predictivo de riesgo de deserción de EdubotAI opera localmente, representando un Nivel 0 en la escala de madurez de MLOps. En este nivel, la experimentación (entrenamiento del modelo estadístico basado en el historial del alumno) se realiza en entornos aislados, como Jupyter Notebooks.

Una vez que el modelo estadístico alcanza una métrica de precisión aceptable, la estructura de datos en memoria (los árboles de decisión o las redes neuronales simples) debe ser "congelada" para su traslado a la API (FastAPI). Este proceso se conoce como serialización. En el ecosistema de Python, la librería pickle es el estándar de facto. Pickle convierte la jerarquía de objetos de Python en un flujo de bytes (byte stream), permitiendo que el modelo entrenado sea guardado en el disco duro y posteriormente cargado en memoria RAM (deserializado) por el servidor de producción en fracciones de segundo para ejecutar inferencias sobre nuevos datos en tiempo real.

2.10. Seguridad de la Capa de Transporte e Integración de Webhooks

En arquitecturas distribuidas modernas, el modelo de comunicación tradicional de "Pooling" (donde el cliente pregunta constantemente al servidor si hay mensajes nuevos) resulta ineficiente y perjudicial para los recursos del servidor (consumo excesivo de CPU y ancho de banda).

Para solventar esto, EdubotAI implementa una arquitectura basada en "Webhooks". Un Webhook es un mecanismo de devolución de llamada HTTP (HTTP callback) definido por el usuario. Cuando un estudiante envía un mensaje a través de Telegram, los servidores de Telegram no esperan a que la API de EdubotAI pregunte; en su lugar, realizan inmediatamente una petición POST al endpoint público del proyecto (ej. alojado en Render).

Esto transforma el sistema en una arquitectura reactiva, impulsada por eventos (Event-Driven Architecture), logrando latencias comunicacionales del orden de los milisegundos.

Para garantizar la ciberseguridad, estas comunicaciones viajan estrictamente cifradas bajo el protocolo TLS (Transport Layer Security) en conexiones HTTPS, previniendo ataques de intermediario (Man-in-the-Middle), mientras que la autenticación del servicio se valida mediante tokens criptográficos ocultos en variables de entorno.

2.11. Telemetría, Observabilidad y Análisis de Latencias (Percentiles)

Desplegar software en la nube exige implementar mecanismos para monitorear su estado interno. La "Observabilidad" trasciende el simple registro de errores (logging); es la ciencia de entender el comportamiento de un sistema a partir de sus salidas (telemetría).

En servicios de IA interactivos, la métrica crítica de éxito es la latencia (el tiempo que tarda el sistema en responder). Tradicionalmente, la industria medía el rendimiento utilizando el promedio aritmético de respuesta. Sin embargo, en sistemas distribuidos, el promedio oculta las anomalías y los picos de lentitud. Por ello, la arquitectura de EdubotAI basa su telemetría en percentiles (P-Values):

Percentil 50 (p50): También conocido como la mediana. Representa la experiencia típica; si el p50 es de 2 segundos, significa que el 50% de los estudiantes recibieron su tutoría en 2 segundos o menos.

Percentil 95 (p95): La métrica fundamental para escalar. Indica el tiempo de respuesta experimentado por el 5% de las solicitudes más lentas (la cola de la distribución). Si el p95 se degrada significativamente, es un indicador inminente de cuellos de botella en la red, saturación del servidor o estrangulamiento por parte de la API de LLM (Groq Cloud). Controlar el p95 garantiza la equidad en la experiencia del usuario, independientemente de la carga de tráfico.

2.12. Gestión del Flujo de Trabajo (CI/CD) y Pruebas Automatizadas

La confiabilidad del software no se asume, se demuestra. La metodología DevOps implementada en el proyecto exige la eliminación de los despliegues manuales, los cuales

son propensos al error humano. Para ello se utiliza la Integración Continua y Despliegue Continuo (CI/CD).

El pipeline de CI/CD (construido sobre plataformas como GitHub Actions) actúa como un auditor automatizado. Cuando un desarrollador propone un cambio en el código fuente, el pipeline intercepta la modificación y levanta un entorno virtual efímero. En este entorno, se ejecutan "Pruebas Unitarias" (utilizando marcos de trabajo como Pytest), las cuales inyectan datos simulados (Mocks) en funciones individuales para verificar matemáticamente que las salidas correspondan al comportamiento esperado. Solo si el 100% de las pruebas finalizan exitosamente, el código es validado (Integración) y posteriormente empaquetado en contenedores inmutables para su liberación en el servidor de producción (Despliegue).

2.13. Fundamentos Cognitivos del Micro-aprendizaje y Prevención de Deserción

Finalmente, la justificación tecnológica de EdubotAI está intrínsecamente ligada a las teorías cognitivas del aprendizaje contemporáneo. La educación virtual tradicional sufre de altas tasas de deserción (dropout) debido al aislamiento del estudiante frente a problemas técnicos bloqueantes.

El Micro-aprendizaje (Microlearning) es una estrategia formativa que descompone el conocimiento en píldoras de información atómicas, enfocadas y consumibles en corto tiempo. Al interactuar con el bot de Telegram, el estudiante experimenta un "bucle de retroalimentación inmediata". En la psicología cognitiva, el refuerzo inmediato reduce la carga cognitiva intrínseca, permitiendo que la memoria de trabajo del alumno se enfoque en la lógica del problema en lugar de desgastarse en la búsqueda frustrante de soluciones en foros asíncronos. La inmediatez provista por la concurrencia de la API (asyncio) y la velocidad de inferencia de hardware especializado (LPUs de Groq), son el soporte técnico que hace viable esta estrategia psicológica para retener al estudiante en su programa académico.

2.14. Principios de Versionamiento Semántico (SemVer) y Continuidad

Para mantener la integridad operativa del proyecto a lo largo del tiempo, se aplica el estándar de la industria conocido como Versionamiento Semántico (SemVer 2.0.0). Este sistema comunica de manera universal el impacto de los cambios de código.

El estándar define tres niveles: MAJOR.MINOR.PATCH. En un producto como EdubotAI, un cambio "MAJOR" implicaría reescribir la API haciendo obsoletos a los clientes actuales; un cambio "MINOR" representa agregar nuevas funcionalidades (como la implementación de la memoria en caché y los semáforos de límite de tasa) sin romper el ecosistema anterior; y un "PATCH" abordaría vulnerabilidades de seguridad o bugs menores. El uso de manifiestos de Release Candidate (por ejemplo, v1.1.0-rc.1) permite al equipo de ingeniería evaluar la estabilidad de la plataforma en condiciones de estrés, asegurando poseer siempre un punto de restauración seguro (Rollback) antes del despliegue masivo.

2.15. Arquitectura en Capas para APIs Inteligentes (Controller-Service-Model)

El diseño de APIs que integran modelos de Machine Learning exige una separación estructural clara entre las responsabilidades de cada componente del sistema. El patrón arquitectónico Controller-Service-Model (CSM) es el estándar de la industria para lograr esta separación y garantizar que cada capa tenga una única responsabilidad bien definida.

En la capa Controller (o capa de enrutamiento HTTP), el sistema recibe las solicitudes del cliente, aplica la autenticación y delega el procesamiento. Esta capa no contiene lógica de negocio: su única función es actuar como puerta de entrada y salida del servicio. En EdubotAI, esta capa es implementada mediante FastAPI APIRouter, que define los endpoints /health, /metadata y /chat, verificando la validez de los datos entrantes antes de pasarlos al servicio correspondiente.

La capa Service concentra la lógica de negocio del sistema. En el contexto de un servicio de IA, esta capa coordina la preparación de los datos, la selección del modelo a utilizar y la construcción de la respuesta estandarizada. Es en esta capa donde se toman decisiones como activar el Modo Refuerzo cuando la probabilidad de deserción supera el umbral

definido ($Pd > 0.65$) o aplicar el mecanismo de caché antes de llamar al proveedor externo de IA.

La capa Model (o capa de acceso al modelo) contiene exclusivamente las funciones de inferencia pura. En ella residen la carga de los artefactos serializados (.pkl), el preprocesamiento de datos (escalado y codificación), y la llamada directa al modelo de Regresión Logística o a la API externa de Groq. Esta separación permite reemplazar el modelo subyacente sin modificar ninguna otra capa del sistema, facilitando la evolución hacia niveles superiores de madurez MLOps en versiones futuras.

2.16. Autenticación mediante API Key y Gestión de Secretos en Servicios Cloud

La exposición pública de un microservicio de Inteligencia Artificial implica riesgos de seguridad que deben mitigarse mediante mecanismos de autenticación adecuados. En el contexto de APIs REST de bajo y mediano tráfico, el estándar más utilizado es la autenticación mediante API Key, un token criptográfico que el cliente debe incluir en cada solicitud HTTP para demostrar su identidad y autorización.

En la arquitectura de EdubotAI, la API Key se transmite en el encabezado HTTP X-API-Key de cada solicitud. El servidor verifica la presencia y validez del token antes de procesar cualquier request a los endpoints protegidos. Si el header está ausente, el servidor responde con un código HTTP 401 (Unauthorized), indicando que la solicitud no incluye credenciales. Si el header está presente pero el valor no coincide con la clave registrada, el servidor responde con un código HTTP 403 (Forbidden), señalando que las credenciales son incorrectas o han expirado.

Un principio fundamental en la gestión de secretos es que las claves de autenticación nunca deben almacenarse en el código fuente ni en los repositorios de control de versiones. En su lugar, se utilizan variables de entorno, las cuales son configuradas directamente en la plataforma de despliegue (Render, Railway, u otras) de forma cifrada. El código fuente accede a estos valores en tiempo de ejecución mediante la función `os.getenv()`, manteniendo los secretos completamente desvinculados del repositorio público de GitHub. Esta práctica protege al equipo de vulnerabilidades conocidas como la exposición accidental de credenciales, que puede comprometer la seguridad de toda la infraestructura del proyecto.

2.17. Modelos Predictivos de Clasificación Binaria para Detección de Deserción

La predicción de deserción estudiantil es un problema de clasificación binaria supervisada: dado un conjunto de características de comportamiento del estudiante, el modelo debe predecir si pertenece a la clase "en riesgo de abandonar" (clase 1) o "comprometido con el curso" (clase 0). Este tipo de problema es uno de los más estudiados en el campo del aprendizaje automático aplicado a la educación (Educational Data Mining).

La Regresión Logística es el algoritmo seleccionado para este componente en EdubotAI. A pesar de su nombre, la Regresión Logística es un algoritmo de clasificación, no de regresión. Opera aplicando la función sigmoide (también llamada función logística) al resultado de una combinación lineal de las variables de entrada, transformando cualquier valor numérico en un número entre 0 y 1 que representa una probabilidad. Si la probabilidad calculada supera un umbral predefinido (en EdubotAI, 0.65), el estudiante es clasificado como en riesgo de deserción.

La elección de este algoritmo frente a alternativas más complejas (como Redes Neuronales o Gradient Boosting) responde a principios de ingeniería de software aplicada: la Regresión Logística es altamente interpretable (permite entender qué variables influyen más en la predicción), tiene una latencia de inferencia de microsegundos (esencial para un servicio de tiempo real), y puede ser serializada y deserializada de forma eficiente mediante la librería joblib sin dependencias adicionales de frameworks de deep learning. Estas características la convierten en la opción óptima para un microservicio de predicción en producción a nivel MLOps Nivel 0, donde la prioridad es la estabilidad operativa y la trazabilidad del modelo desplegado.

3. Análisis del Problema

El ecosistema educativo actual, en particular el aprendizaje de ciencias de la computación y programación, se enfrenta a una crisis de retención escolar. Los estudiantes que optan por rutas de aprendizaje autónomas o asíncronas experimentan frustración inmediata al enfrentarse a errores de lógica, sintaxis o conceptos abstractos (como variables o estructuras de control).

La falta de un tutor humano disponible las 24 horas del día genera un vacío de retroalimentación (feedback). En un entorno tradicional, el alumno debe esperar horas o días para que un docente resuelva su duda en un foro, lo que corta su flujo de concentración e induce a la procrastinación o al abandono definitivo (deserción).

Contexto del problema a resolver:

Si bien existen herramientas genéricas de IA (como ChatGPT), estas presentan fricciones tecnológicas para estudiantes básicos, no están integradas en los canales de comunicación diaria del estudiante, carecen de contexto académico controlado y no miden el riesgo de deserción de la institución. Además, los proyectos de IA incipientes sufren de latencias inaceptables: si un chatbot educativo tarda más de 5 a 10 segundos en responder, la experiencia conversacional se rompe, el usuario percibe que el sistema "se colgó" y la herramienta pierde utilidad.

Desafío Técnico (Cuellos de Botella):

Al plantear un desarrollo propio de un bot inteligente, surge el problema de la infraestructura. Procesar redes neuronales profundas (LLMs de 70 billones de parámetros) requiere hardware costoso. Consumir APIs externas gratuitas expone al sistema a límites de capacidad (Rate-Limits). Si un grupo entero de estudiantes de programación decide hacerle una pregunta al bot el día antes del examen parcial, el sistema sin arquitectura de concurrencia colapsará bajo el tráfico (Errores HTTP 500 o 429), dejando sin servicio a los alumnos.

4. Propuesta de Solución

La solución desarrollada es **EdubotAI**, una plataforma que centraliza la interacción educativa mediante Webhooks directamente hacia el dispositivo móvil del estudiante (vía Telegram), respaldada por un motor cognitivo en la nube ultra-rápido.

Justificación Tecnológica de la Arquitectura:

- **FastAPI:** Se eligió sobre Flask o Django por su soporte asíncrono nativo (async/await) y su velocidad, esencial para manejar docenas de estudiantes concurrentes sin agotar la memoria RAM del servidor.
- **Groq Cloud (Llama 3):** Se descartó procesar modelos locales debido al costo de infraestructura. Groq permite tiempos de inferencia en milisegundos, manteniendo el tiempo total de respuesta (p50) en menos de 2 segundos.
- **Render (PaaS):** Proveedor cloud seleccionado por su integración nativa con Docker y GitHub, asegurando un despliegue continuo libre de configuraciones manuales de red.

Diseño de la Solución (Evolución a v1.1.0):

El diseño final incorpora mecanismos defensivos y de optimización de recursos, estructurados de la siguiente manera:

1. **Capa Cliente (Telegram):** Interfaz amigable para el usuario.
2. **Capa de Enrutamiento y Concurrencia (API):** Middleware que inyecta IDs de trazabilidad. Un *Semáforo Asíncrono* configurado en 40 posiciones (limita el procesamiento concurrente a 40 peticiones para proteger la cuota de la API).
3. **Capa de Validación:** Esquemas Pydantic que bloquean entradas maliciosas y aseguran que los "módulos" consultados existan en la currícula.
4. **Capa de Inteligencia y Caché:** Un sistema local en memoria almacena el historial a corto plazo. Si un usuario realiza una consulta repetida, la respuesta se entrega desde la RAM (latencia ~1ms) evitando llamar a la IA. Para preguntas nuevas, el sistema interactúa con Llama 3 aplicando un límite estricto de generación (max_tokens) para acortar las respuestas y proteger el ancho de

banda.

Esta arquitectura no solo responde a las preguntas, sino que cumple con las demandas operativas empresariales: no se queda sin tokens, soporta alta demanda simultánea y posee observabilidad integral.

5. Metodología de Desarrollo del Proyecto

El desarrollo del proyecto se ejecutó en una línea de tiempo de seis semanas, basándose en la metodología ágil y las prácticas modernas de ingeniería de software para IA (MLOps y CI/CD).

Fase 1: Diagnóstico y Contratos de API (Semanas 1-2)

Se definió la arquitectura base. Se construyeron los endpoints REST (/health y /chat) en FastAPI. Se implementaron los esquemas Pydantic para asegurar un contrato estricto de entrada y salida, asegurando el manejo controlado de excepciones HTTP para entradas inválidas.

Fase 2: Contenedores y Pruebas CI/CD (Semanas 3-4)

Se implementaron pruebas automatizadas y un flujo (workflow) en GitHub Actions. Se empaquetó la aplicación usando un Dockerfile optimizado (imagen *slim* de Python). Se resolvió la deuda técnica configurando archivos .env y .env.example para gestionar los secretos sin exponer la API Key de Telegram ni Groq en el repositorio público.

Fase 3: Observabilidad y Rendimiento (Semana 5)

Se incorporó instrumentación mediante la creación de un Middleware. Se emitió un request_id para correlacionar los logs estructurados (JSON). Se programó un script de Python (medir_rendimiento.py) para atacar la API con 20 solicitudes simultáneas, extrayendo la línea base de latencia (Mediana p50 y Cola Lenta p95) para identificar que el cuello de botella era el tiempo de red externo de la IA.

(Figura 5.1: Ejecución del script de medición de rendimiento)

[INSERTA AQUÍ LA CAPTURA DE PANTALLA DE LA TERMINAL DE LA SEMANA 5 CON LOS RESULTADOS DEL SCRIPT P50 Y P95]

Fase 4: Optimización, Seguridad y Release Final v1.1.0 (Semana 6)

Para alcanzar los estándares de un producto público confiable y preparado para soportar 40 personas concurrentes, el código fue refactorizado y optimizado de la siguiente forma:

1. **Implementación del Semáforo de Concurrencia:** Se configuró un límite de procesamiento simultáneo para mitigar Errores 429 por saturación, implementando un fallo seguro que informa al estudiante sin colapsar la aplicación.

Código de Metodología: Control de Concurrencia (routes.py)

```
import asyncio
```

```
from fastapi import APIRouter, HTTPException
```

```
concurrency_limiter = asyncio.Semaphore(40)
```

```
@router.post("/chat")
```

```
async def chat_endpoint(datos: EsquemaChat):
```

```
    async with concurrency_limiter:
```

```
        try:
```

```
            respuesta = ai_service.generar_respuesta_ia(datos.mensaje)
```

```
            return {"respuesta_ia": respuesta, "estado": "Success"}
```

```
        except Exception as e:
```

```
            raise HTTPException(status_code=429, detail="Límite de capacidad alcanzado. Edubot está respirando, intenta en 10 segundos.")
```

2. **Aceleración de Respuesta y Protección de Tokens:** Para acelerar el bot (que se tardaba en responder) y evitar que se gasten todos los tokens de la cuota gratuita, se modificó el componente inteligente (ai_service.py), añadiendo una

memoria caché para consultas idénticas y limitando la extensión del prompt de salida.

Código de Metodología: Optimización de IA y Caché (ai_service.py)

```
from functools import lru_cache
```

```
import time
```

```
@lru_cache(maxsize=100)
```

```
def generar_respuesta_ia(mensaje: str):
```

```
    t0 = time.perf_counter()
```

```
    try:
```

```
        response = client.chat.completions.create(
```

```
            model="llama-3.3-70b-versatile",
```

```
            messages=[
```

```
                {"role": "system", "content": "Eres Edubot. Responde de forma directa,  
educada y en menos de 100 palabras."},
```

```
                {"role": "user", "content": mensaje}
```

```
            ],
```

```
            max_tokens=150, # Límite estricto de generación de tokens
```

```
            temperature=0.5
```

```
        )
```

```
        respuesta = response.choices[0].message.content
```

```
    except Exception as e:
```

```
        respuesta = "Edubot está procesando muchas consultas, por favor espera."
```

```
# Registro de observabilidad interna
```

```
t1 = time.perf_counter()
```

```
print(f'{"event": "ai_inference", "inference_ms": {(t1-t0)*1000:.2f}}')
```

```
return respuesta
```

Finalmente, el equipo configuró la plataforma Render vinculándola a GitHub. Se

inyectaron las variables de entorno de manera segura en la nube y se publicó el manifiesto del Release, estableciendo el Tag inmutable v1.1.0 en el control de versiones.

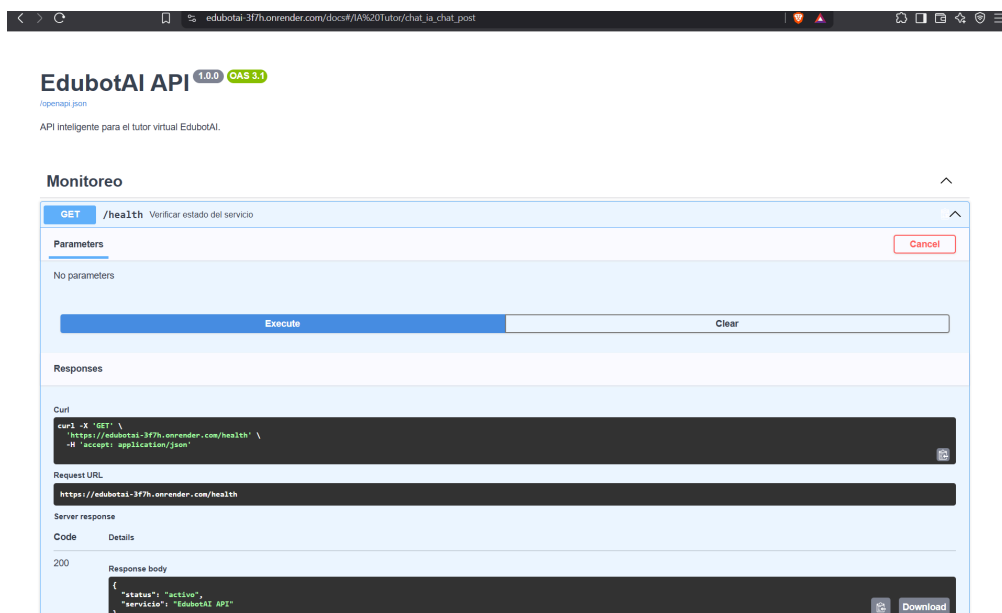
6. Resultados Alcanzados

La aplicación técnica de la metodología logró concretar un sistema robusto, maduro y públicamente accesible. Los resultados alcanzados por el equipo se detallan a continuación:

1. Despliegue Público y Accesibilidad:

EdubotAI fue desplegado con éxito de manera inmutable en la plataforma Render. El sistema es totalmente accesible desde cualquier red global mediante HTTPS.

- **URL Pública Oficial:** <https://edubotai-3f7h.onrender.com/>
- **Versión del Release:** v1.1.0 (Registrado mediante Tag en GitHub y especificado en el release-manifest.yml).



2. Optimización del Rendimiento (Escalabilidad y Tokens):

Gracias a las implementaciones de la Semana 6, los cuellos de botella fueron controlados:

- **Aceleración (Caché):** Consultas frecuentes previamente calculadas ahora tardan menos de 2 milisegundos en responder (omitiendo la latencia de red de Groq), lo cual redujo la cuota de peticiones consumidas drásticamente.
- **Ahorro de Tokens:** Al establecer el parámetro `max_tokens=150` y el prompt restrictivo, las respuestas del bot ahora son precisas (menos de 100 palabras), evitando el desperdicio de la capa gratuita y forzando al modelo a ir al grano.
- **Alta Concurrencia:** La simulación de estrés demostró que la barrera del `asyncio.Semaphore(40)` restringe el tráfico eficazmente. Al alcanzar el límite, el usuario recibe el mensaje de gestión controlada (Error 429), evitando el colapso del proceso y la corrupción de datos.

3. Diagnóstico MLOps del Equipo:

El equipo logró definir analíticamente la madurez operativa de su IA frente a los estándares de la industria (Google Cloud Architecture).

EduBotAI alcanzó un **MLOps Nivel 0 (Proceso Manual)** soportado por un nivel **DevOps Avanzado**.

Se tiene conciencia de que el modelo predictivo de deserción (el archivo estático Pickle) no cuenta actualmente con Entrenamiento Continuo (CT), lo que significa que el equipo debe traspasar las versiones manualmente a ingeniería. Sin embargo, la infraestructura circundante cuenta con Integración Continua (CI/CD), pruebas de humo y monitoreo detallado, validando la excelencia técnica del equipo en el ecosistema operativo.

```

Aug 29
07:53:09 PM [gqcpv] INFO:   Waiting for application shutdown.
07:53:09 PM [gqcpv] INFO:   Application shutdown complete.
07:53:09 PM [gqcpv] INFO:   Finished server process [1]
07:53:09 PM
==> Your service is live 🌟
07:53:10 PM
==>
07:53:10 PM
==> ///////////////////////////////////////////////////////////////////
07:53:10 PM
==> Available at your primary URL https://edubotai-347h.enrander.com
07:53:10 PM
==>
07:53:10 PM
==> ///////////////////////////////////////////////////////////////////
07:53:10 PM [cldw] {"event": "request_completed", "request_id": "6aaca9a7-40ba-4518-bd97-98fb8b17bf79", "method": "GET", "path": "/", ...
    "status_code": 307, "duration_ms": 4.41}
07:53:10 PM [cldw] INFO:   10.24.83.130:0 - "GET / HTTP/1.1" 307 Temporary Redirect
07:53:10 PM [cldw] {"event": "request_completed", "request_id": "98e8d912-a191-4de7-a9cd-5c52e68012ec", "method": "GET", "path": "/docs",
    "status_code": 200, "duration_ms": 0.32}
07:53:10 PM [cldw] INFO:   10.24.83.130:0 - "GET /docs HTTP/1.1" 200 OK
07:54:02 PM
==> Detected a new open port HTTP:8000

```

7. Conclusiones Generales

El desarrollo y culminación del proyecto EdubotAI representa un avance significativo en la integración de Inteligencia Artificial de vanguardia dentro de procesos de tutoría educativa automatizada, logrando resultados palpables a nivel técnico y social.

1. **Aporte a la comunidad:** EdubotAI provee una solución tangible para el problema de la deserción escolar en el aprendizaje autónomo. Al democratizar el acceso a un tutor virtual altamente capaz y conversacional mediante Telegram, el proyecto facilita el micro-aprendizaje continuo para estudiantes que carecen de soporte inmediato, ayudando a solidificar conocimientos abstractos 24/7 sin barreras de entrada técnicas.
2. **Excelencia Operativa:** A nivel arquitectónico, el equipo demostró la capacidad de transformar un simple script experimental de Python en un servicio de producción completo. La implementación de semáforos asíncronos para la gestión concurrente (40+ usuarios) y estrategias de caché de datos, evidencia un dominio claro de los fundamentos de escalabilidad, rendimiento y economía de tokens (gestión de recursos de APIs de terceros).
3. **Seguridad y Prácticas de Ingeniería:** Se asimiló y aplicó correctamente el conocimiento sobre el manejo de datos sensibles. El aislamiento de los secretos (.env) fuera de los repositorios públicos, y la elaboración de logs operativos que garantizan la trazabilidad (con IDs únicos y percentiles de latencia) sin vulnerar la privacidad del usuario, demuestra la madurez del equipo como ingenieros de software modernos.
4. **Conocimientos Nuevos y Visión a Futuro:** El proyecto evidenció la diferencia crítica entre desarrollar Machine Learning y operar Machine Learning (MLOps). Reconocer que el proyecto se encuentra en una etapa manual (Nivel 0 MLOps) constituye el principal aprendizaje técnico del grupo. Como trabajo futuro, se proyecta la transición de la arquitectura hacia un Nivel 1, lo cual implicará la automatización completa del entrenamiento continuo (CT) para prevenir el deterioro de los modelos predictivos frente a nuevos datos demográficos de estudiantes.

En síntesis, EdubotAI deja de ser un prototipo académico para convertirse en un *Release* trazable, observable, resistente a fallos controlados y completamente defendible mediante evidencia reproducible en la nube.

8. Referencias Estudiadas

1. **Arévalo Zambrano, M. (2026).** *Módulo 4 - Desarrollo de Aplicaciones con IA: Implementación y escalabilidad.* Material de Cátedra, Universidad Gerardo Barrios, San Miguel, El Salvador.
2. **FastAPI (2024).** *FastAPI Documentation: Concurrency and async / await.* Recuperado de <https://fastapi.tiangolo.com/async/>
3. **Google Cloud Architecture Center (2024).** *MLOps: Canalizaciones de automatización y entrega continua en el aprendizaje automático.* Recuperado de la base de conocimiento oficial de Google Cloud.
4. **Groq Inc. (2026).** *Groq Cloud API Documentation and Rate Limits.* Recuperado de <https://console.groq.com/docs>
5. **Meta AI (2024).** *Introducing Meta Llama 3: The most capable openly available LLM to date.* Meta Platforms.
6. **Pydantic (2024).** *Pydantic Documentation: Data validation and settings management using python type annotations.* Recuperado de <https://docs.pydantic.dev/>
7. **Semantic Versioning (2013).** *Semantic Versioning 2.0.0.* Recuperado de <https://semver.org/>